

Quarto Guide

Jannik Korbach

This document demonstrates the core features of Quarto.

Document Structure

A Quarto Markdown Document (.qmd) consists of two parts: a YAML header and a body [1].

The YAML header is enclosed by three dashes (---) at the top of the file. It defines metadata and rendering options using key-value pairs [2]:

```
---  
title: "My Report"  
author: "Your Name"  
date: today  
format: pdf  
---
```

Visit the official Quarto documentation for more header properties [3].

Project Configuration with _quarto.yml

Header properties can also be defined inside a _quarto.yml file inside the root of the project. Any document in the same folder automatically inherits these settings [3].

Document-level YAML always overrides _quarto.yml. Use the header for document-specific settings and _quarto.yml for global default settings [3].

Code Cells

Code cells are defined with triple backticks and a language identifier in braces. Cell options are set with `#|` comments [2]:

```
#| label: fig-example
#| fig-cap: "Simple line plot"
#| echo: false

import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [1, 4, 9])
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

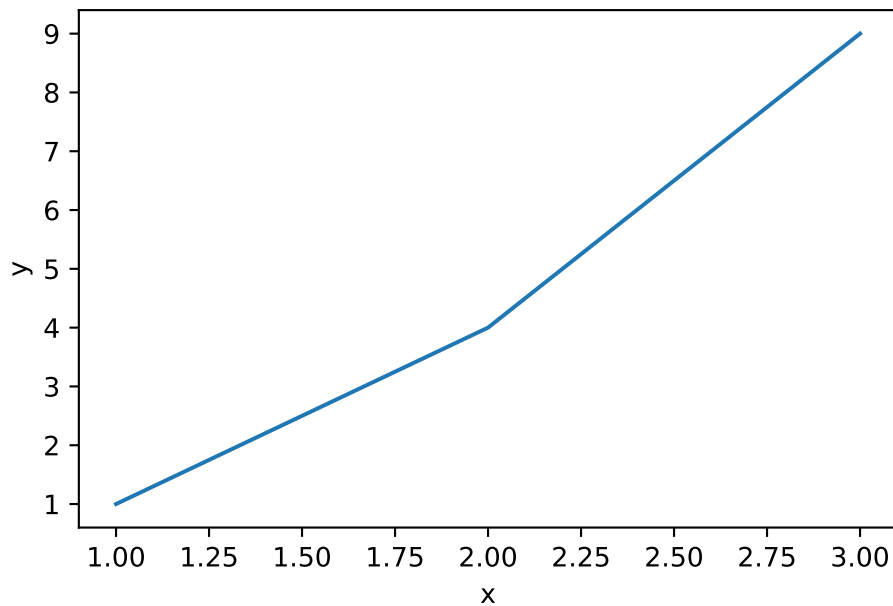


Figure 1: Simple line plot

Relevant cell options:

- `echo: false`: only show output and hide the code
- `eval: false`: show code without executing it
- `label: xy`: make figure cross-referencable

Cross References

Labels enable automatic numbering and hyperlinks. Use `@label` in text to reference the element [2].

Label prefixes define the element type:

- `fig-`: Figure
- `tbl-`: Table
- `eq-`: Equation
- `eq-`: Section

Citations with BibTeX

Add a `.bib` file to your YAML header and cite using `@key` or `[@key]`:

```
bibliography: references.bib
```

The bibliography is generated by Quarto at the end of the document. You can add a header at the end of the document if you want to have a header for the bibliography [3].

Setting up Reproducible Project

This section provides a workflow for setting up a Python Quarto project with reproducibility in mind. It takes recommendations for Quarto with R [4] and the official Quarto guide [3].

Create the Project

```
quarto create project
```

Choose `default` as the project type and give it a name. This creates a folder with a `_quarto.yml` file and a starter `.qmd` file [3].

Create a Virtual Environment

Virtual environments isolates the project's dependencies from other projects.

```
python -m venv .venv
source .venv/bin/activate # Mac/Linux
.venv\Scripts\activate # Windows
```

Install Dependencies & Pin Versions

Install the required packages for the project, then freeze the versions into a `requirements.txt` file:

```
pip install jupyter ipykernel pandas # all your required packages
pip freeze > requirements.txt
```

Other users can reproduce the environment later on with this command:

```
pip install -r requirements.txt
```

References

- [1] M. Çetinkaya-Rundel and C. Wickham, “Quarto: The practical guide.” [Online]. Available: <https://quarto-tdg.org/>
- [2] S. Rieger, “A not so short introduction to quarto.” [Online]. Available: <https://riegersv.en.github.io/library/software/introduction-to-quarto.html>
- [3] Quarto, “Guide.” [Online]. Available: <https://quarto.org/docs/guide/>
- [4] E. Le Foll, “Quarto for reproducible research workflows and academic publishing.” [Online]. Available: <https://elenlefol.quarto.pub/quarto4research/#sec-Reproducibility>